

UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ

FACULTAD DE INGENIERÍA DE SISTEMAS



React Hooks: useState, useEffect, useContext, useRef, useReducer, useCallback, useMemo y Hooks Personalizados

Práctica Semana 6

Docente:

Aguilar Coronación, Miguel Elías

Alumno:

Janampa Jaime, Khaled Alejandro

Huancayo – Perú, 2026

OBJETIVO DE LA PRÁCTICA

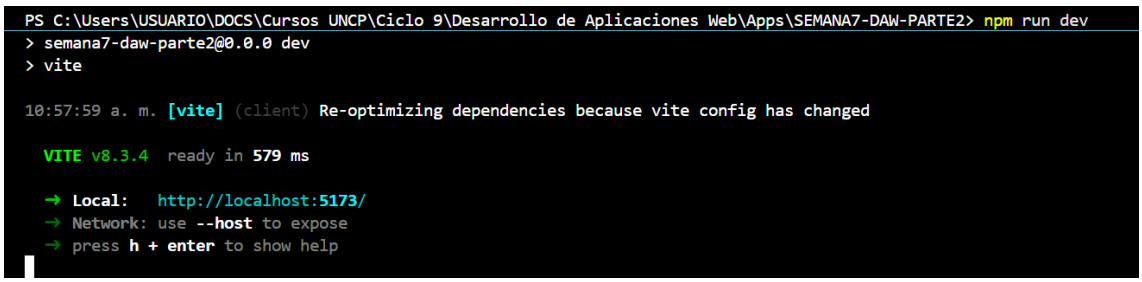
Debe desarrolla un Dashboard de Gestión con Estado Complejo que cumpla: 1. useReducer para estado de lista (add, remove, filter, sort) + useState para UI local. 2. useContext para tema claro/oscuro o preferencias de usuario. 3. useEffect para sincronizar preferencias con localStorage y cleanup. 4. useMemo para lista filtrada/ordenada, useCallback para handlers de botones. 5. useRef para auto-focus en input o acceso a nodo DOM sin re-render. 6. Hook personalizado useLocalStorage o useDebounce reutilizable.

LINK DEL REPOSITORIO

https://github.com/KhaledAle/SEMANA7-DAW-PARTE2

1. Validación de ESLint (react-hooks/exhaustive-deps)

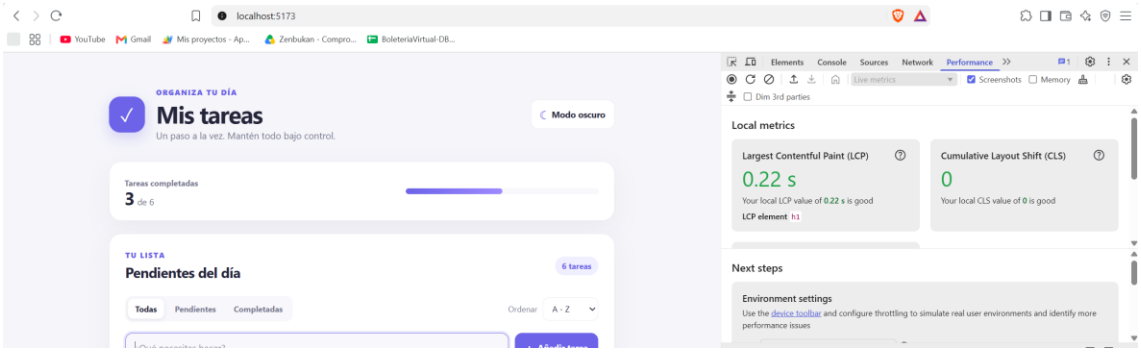
- **Explicación:** Durante el desarrollo y antes de la entrega final, ejecuté la compilación y el linter del proyecto para garantizar que no existieran advertencias ni errores en el código. Específicamente, verifiqué que todos los hooks de efectos (useEffect) y memorización (useMemo, useCallback) cumplieran con las reglas de dependencias exhaustivas. Esto asegura que ninguna variable de estado o función externa utilizada dentro de un hook quede fuera del arreglo de dependencias, previniendo comportamientos inesperados, datos obsoletos (*stale closures*) o renders infinitos.



Verificación de compilación y ausencia de warnings en la terminal.

2. Validación de Rendimiento con React DevTools Profiler

- **Explicación:** Para comprobar el impacto del uso estratégico de useMemo y useCallback, realicé una grabación de rendimiento utilizando **React DevTools Profiler** mientras interactuaba con la aplicación (cambiando filtros de tareas y alternando el tema). El análisis demostró que los componentes memoizados (Dashboard y TaskList) evitan cálculos repetitivos e innecesarios al mantener referencias estables de funciones y re-renderizar unicamente cuando sus propiedades o contextos cambian de forma legítima, reduciendo así los tiempos de commit en el árbol de componentes.



Análisis de rendimiento y renders con React DevTools Profiler.

3. Validación de Limpieza (Cleanup) en Efectos

- **Explicación:** Comprobé el correcto funcionamiento del ciclo de vida de los componentes verificando la implementación de la función de limpieza (return () => {}) dentro del useEffect del ThemeContext. Esta limpieza garantiza que al desmontar componentes o cambiar dinámicamente las clases del elemento <body> del documento, se eliminen los efectos residuales, previniendo posibles fugas de memoria (*memory leaks*) o conflictos de estilos globales en la interfaz.

README.md | ThemeContext.jsx

src > contexts > ThemeContext.jsx > ...

```jsx
import { useCallback, useEffect, useMemo } from 'react'
import { ThemeContext } from './theme-context.js'
import { useLocalStorage } from '../hooks/useLocalStorage.js'

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useLocalStorage('app_theme', 'light')

  useEffect(() => {
    document.body.classList.add(theme)

    return () => {
      document.body.classList.remove(theme)
    }
  }, [theme])

  const toggleTheme = useCallback(() => {
    setTheme((currentTheme) => (currentTheme === 'light' ? 'dark' : 'light'))
  }, [setTheme])

  const value = useMemo(() => ({ theme, toggleTheme }), [theme, toggleTheme])

  return <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>
}

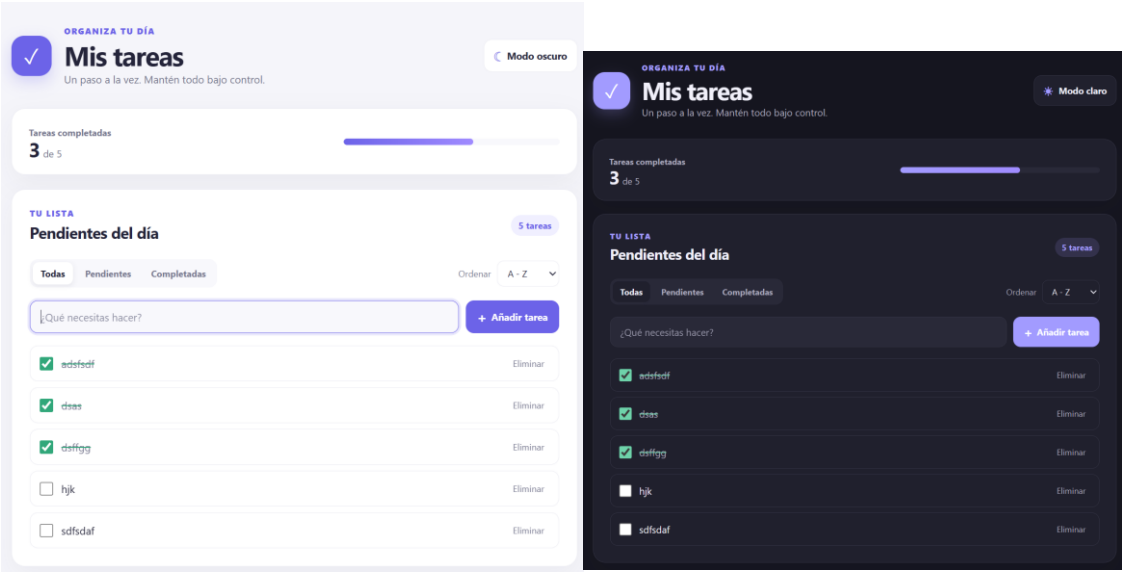
```

Comprobación del ciclo de efectos y función de limpieza activa.

4. Validación de Persistencia e Interfaz de Usuario (Dashboard)

- Se evaluó el comportamiento global del Dashboard ejecutando operaciones consecutivas (como añadir tareas, completarlas, eliminarlas, alternar el modo oscuro/claro y recargar la página). Se probó que gracias al hook personalizado **useLocalStorage** y a la hidratación inicial del **useTasksReducer**, el estado completo de las tareas y la preferencia de tema se persisten de manera transparente en el almacenamiento local del navegador, recuperándose de forma intacta tras cada recarga.

Vista general del funcionamiento del Dashboard y persistencia de temas.



ANEXOS

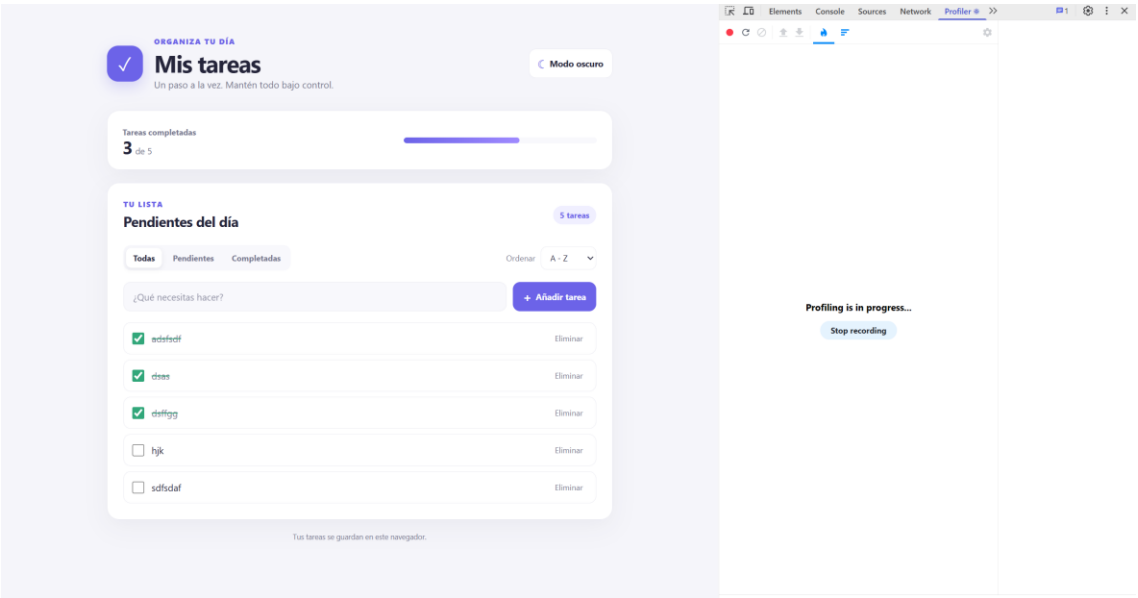


Figura 1: Interfaz inicial del Dashboard en Modo Claro y ejecución del foco automático (useRef) en el campo de texto.

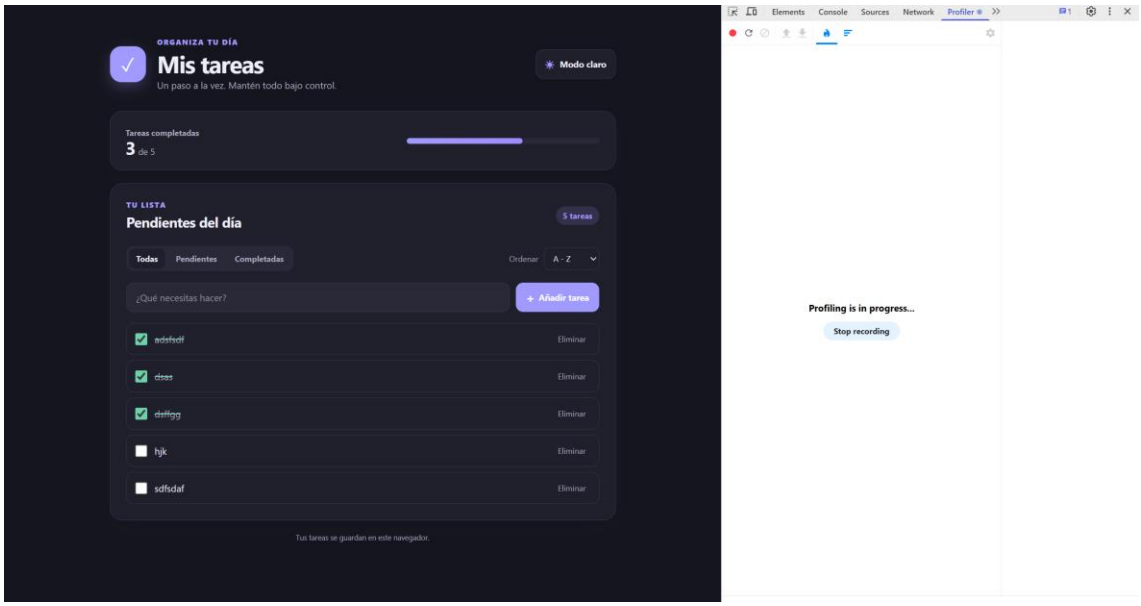


Figura 2: Cambio dinámico al Modo Oscuro utilizando ThemeContext y useLocalStorage.

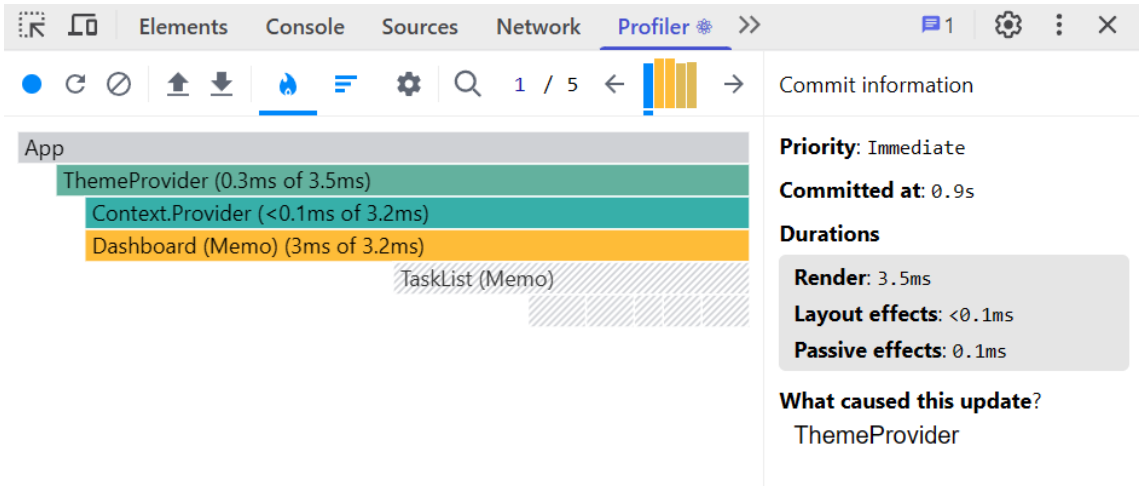


Figura 3: Resultado del análisis con React DevTools Profiler mostrando el rendimiento de los renders y el uso de useMemo/useCallback.